



UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y
NATURALES

SÍNTESIS DE REDES ACTIVAS

Tarea Complementaria N°2
Diseño de Filtros Activos con Python

Profesor Titular: Dr. Ing. Ferreyra Pablo

Profesor Adjunto: Ing. Reale Cesar

Alumnos: Dávila Tomassi, Carlos Valentino
Mendes Rosa, Agustín
Monja, Ernesto Joaquín

Año 2025

Índice

1. Introducción	2
1.1. Consigna	2
2. Desarrollo	2
2.1. Introducción a los Filtros Activos	2
2.2. Python como Software de Diseño	3
2.2.1. Pasaje de Especificaciones a Funciones de Transferencia	3
2.2.2. Representación del Sistema	5
2.3. Ejemplo de Aplicación en Python	8
3. Conclusión	12
4. Bibliografía	12
5. Anexo	12

1. Introducción

1.1. Consigna

En este presente informe, se detalla de forma conceptual el funcionamiento de las distintas librerías que cuenta el programa Python para poder sintetizar filtros activos en base a ciertos requerimientos que le presente el programador tal así como se realizo para el Trabajo Práctico N°4 de esta asignatura.

2. Desarrollo

2.1. Introducción a los Filtros Activos

De acuerdo a la bibliografía utilizada, se define a un filtro como una red utilizada para cambiar la forma del espectro de la frecuencia de una señal eléctrica y estos son clasificados de acuerdo a la función que realizan, es decir que forma le dan al espectro de frecuencias.

Como se menciona anteriormente, los filtros se definen de acuerdo a que forma le dan al espectro de las frecuencias lo cual implica un uso de las Transformadas de Laplace y consecuentemente el análisis de funciones de transferencias. De acuerdo a la forma de la función de transferencia que tenga el filtro, es decir sus ceros, polos y constantes, se definirá la función del filtro tal como se presenta en la siguiente figura:

Filter function	Transfer function
Low-pass	$K \frac{1}{s^2 + as + b}$
High-pass	$K \frac{s^2}{s^2 + as + b}$
Band-pass	$K \frac{s}{s^2 + as + b}$
Band-reject	$K \frac{s^2 + d}{s^2 + as + b}$
Delay equalizer	$\frac{s^2 - as + b}{s^2 + as + b}$

Figura 1: Funciones de Transferencia de distintos tipos de filtros

La bibliografía utilizada menciona como sintetizar las funciones de transferencia de distintos filtros en componentes que mediante una topología activa del filtro, definirán los componentes a utilizar, sin embargo no se realiza un análisis de como deducir las funciones

de transferencia de acuerdo a las especificaciones de un filtro (por ejemplo, frecuencias de corte, atenuaciones máximas y mínimas, etc.).

Se busca entonces en este informe, detallar el proceso de como realizar funciones de transferencia de acuerdo a las especificaciones de un filtro utilizando software.

2.2. Python como Software de Diseño

La elección de Python como software de diseño para los filtros se debe a la ventaja que este posee al ser Software Gratuito y de Código Abierto, permite entender los algoritmos que ejecutan los comandos de sus librerías además de ser un lenguaje de programación de uso general lo que no nos limita únicamente a fines matemáticos (como si lo hace Matlab/Octave) o fines electrónicos (como PSpice/LTSpice).

2.2.1. Pasaje de Especificaciones a Funciones de Transferencia

La librería *SciPy* de Python ofrece, entre tantas funciones, el módulo *.signal* el cual se dedica al análisis y procesamiento de señales y sistemas lineales. Este módulo carece de los conocimientos de electrónica utilizados para sintetizar filtros con topologías Sallen-Key por ejemplo, pero si sabe determinar de forma matemática las funciones de transferencia que representan los requerimientos de un filtro, es decir por ejemplo:

- La cantidad de polos que necesita un filtro Butterworth
- La ubicación del Ripple (Banda de Paso o de Rechazo) de un filtro Chebyshev
- La aproximación deseada para un filtro.
- Entre otras funciones

Este módulo ofrece para el diseño de funciones de transferencia, las siguiente funciones claves

Función	Aproximación	Característica Principal de la Aproximación
<i>signal.butter()</i>	Butterworth	Máxima Planicidad en la Banda de Paso.
<i>signal.cheby1()</i>	Chebyshev I	El ripple del Chebyshev se encuentra en la Banda de Paso.
<i>signal.cheby2()</i>	Chebyshev II	El ripple del Chebyshev se encuentra en la Banda de Rechazo.
<i>signal.ellip()</i>	Elíptica	Hay ripple en ambas bandas, pero entre ellas hay una transición mas abrupta.
<i>signal.bessel()</i>	Bessel	Se obtiene Fase casi lineal en la Banda de Paso.

Cuadro 1: Funciones principales del módulo *SciPy*

Estas funciones comparten todas la misma lógica en sus parámetros pero obtienen resultados distintos, donde por ejemplo, si se tomara un Filtro Pasa Banda aproximado por Butterworth, el código a utilizar será:

```

1 import numpy as np
2 from scipy import signal
3
4 wp = [2*np.pi*f1, 2*np.pi*f2]
5 num, den = signal.butter(n = 2, rp = 1, wp, btype='bandpass', analog=True, output = 'I')
6
7 print("Numerador:", num)
8 print("Denominador:", den)

```

Donde los parámetros que toma esta función son:

- num, den: Numerador y denominador de la función de transferencia
- n: Orden del filtro pasa bajos normalizado. Se presenta a continuación, la tabla de transformaciones de filtro pasa bajos normalizados a otros filtros:

Tipo de filtro	Transformación en s	Parámetros	Orden final
Pasa bajos (LP)	$s \rightarrow \frac{s}{\omega_c}$	ω_c : frecuencia de corte	N
Pasa altos (HP)	$s \rightarrow \frac{\omega_c}{s}$	ω_c : frecuencia de corte	N
Pasa banda (BP)	$s \rightarrow \frac{s^2 + \omega_0^2}{BW s}$	$\omega_0 = \sqrt{\omega_1 \omega_2}$	$2N$
Rechaza banda (BS)	$s \rightarrow \frac{BW s}{s^2 + \omega_0^2}$	$\omega_0 = \sqrt{\omega_1 \omega_2}$	$2N$

Cuadro 2: Transformaciones de Frecuencia aplicadas al prototipo Pasa Bajos de orden N

Donde $BW = \omega_2 - \omega_1$ siendo este el ancho de banda angular del filtro pasa banda o rechaza banda

- rp: Ripple en $[dB]$ para la banda de paso (para un filtro rechaza banda, se trata del ripple en la banda de rechazo).
- wp: Frecuencia Crítica Normalizada de los Polos.
- btype: Tipo de Filtro, este puede ser:
 - `btype = 'lowpass'`: Filtro Pasa Bajos.
 - `btype = 'highpass'`: Filtro Pasa Altos.
 - `btype = 'bandpass'`: Filtro Pasa Banda.
 - `btype = 'bandstop'`: Filtro Rechaza Banda.

- analog: Indica si se busca diseñar un filtro en tiempo continuo o discreto.
- output: Se trata de un parámetro opcional el cual me indica el tipo de salida de la definición

Este ejemplo sirve para ilustrar varias características de esta función. Una de ellas es que si bien se definió a un filtro pasa banda de 2^{do} orden, al compilar el código se obtiene una función de transferencia de 4^{to} orden dado su denominador.

Esto se debe a que la librería de Python *SciPy* siempre parte de un prototipo de pasa bajos el cual mediante transformaciones *HP*, *BP* ó *BS* (High-Pass, Band-Pass y Band-Stop del Inglés) lo transforma en el filtro deseado de acuerdo a la Tabla 2. Justamente en este ejemplo, la transformación utilizada es la del pasa banda, la cual duplica el orden del filtro original, ya que cada polo del prototipo pasa bajos se transforma en un par de polos complejos conjugados en el pasa banda; y lo mismo es de esperarse si se utiliza un filtro rechaza banda de las mismas características.

Nótese también que *wp* aquí se definió como un array de números dependientes de la frecuencia angular de los polos. Estas funciones toman como parámetros en este apartado, unicamente valores dados en radianes sobre segundos y no debe utilizarse frecuencias en Hertz.

El último parámetro es de vital importancia ya que este define si se esta diseñando un filtro de forma analógica o de forma digital. Es fundamental entender que no es lo mismo diseñar un filtro en tiempo continuo que en tiempo discreto ya que la herramienta de análisis y traspaso al dominio de la frecuencia cambia, es decir que si en tiempo continuo se utiliza la Transformada de Laplace, para tiempo discreto se utiliza la Transformada Z.

En el anexo 1, se incluye un fragmento de la librería en el cual se documenta que realiza cada parámetro y como funciona el código. Se observa en el Anexo 1, que cada función descrita en la Tabla 1, consiste en un llamado a una función mas general denominada *iirfilter()* en la cual se especifica el tipo de aproximación a diseñar.

2.2.2. Representación del Sistema

Los parámetros que nos entregan las funciones descritas en el apartado anterior son el numerador y denominador de la función de transferencia global del filtro y suelen ser de la forma:

$$T(s) = \frac{b_ms^m + b_{m-1}s^{m-1} + \dots + b_2s^2 + b_1s + b_0}{a_ns^n + a_{n-1}s^{n-1} + \dots + a_2s^2 + a_1s + a_0} \quad (1)$$

Sin embargo, se tiene que según la Figura 1, un filtro no puede ser de orden mayor a 2, por lo tanto, se debe descomponer en funciones de transferencia bicuadráticas al filtro obtenido en el apartado anterior y para ello haremos uso de otras funciones que trae el

módulo `.signal` de la librería *SciPy*. Se presentan a continuación 2 formas de encarar y resolver este problema:

- Cero, Polo y Constante: Esta forma, consiste en tomar la función de transferencia obtenida en el apartado anterior y mediante la función `signal.tf2zpk()` ("Transfer Function to Zero, Pole, K: Constant" del Inglés) obtendremos los polos y ceros de la función. Veamos como aplicar esta función a un ejemplo:

```
1 import numpy as np
2 from scipy import signal
3 import matplotlib.pyplot as plt
4
5 f_c = 1000
6 w_c = 2*np.pi*f_c
7 num, den = signal.butter(2, w_c, btype='lowpass', analog=True)
8 z, p, k = signal.tf2zpk(num, den)
9 print("Ceros: ", z)           %
10 print("Polos: ", p)          % -4442.88 + j4442.88 ; -4442.88 - j4442.88
11 print("Constante: ", k)      % 39478417.60
12
13 w = np.logspace(1, 5, 1000)
14 w, h = signal.freqs(num, den, w)
15
16 plt.semilogx(w/(2*np.pi), 20*np.log10(abs(h)))
17 plt.xlabel("Frecuencia [Hz]")
18 plt.ylabel("Magnitud [dB]")
19 plt.grid()
20 plt.show()
```

En este ejemplo, se diseño un filtro pasa bajos aproximado con una con una frecuencia de corte de 1 $[kHz]$ y el código nos muestra la ubicación de ceros, polos y la constante los cuales pueden tener grandes valores dado el escalado realizado por ω_c .

El resto del código sirve para realizar un diagrama de bode de esta función de transferencia el cual se presenta a continuación:

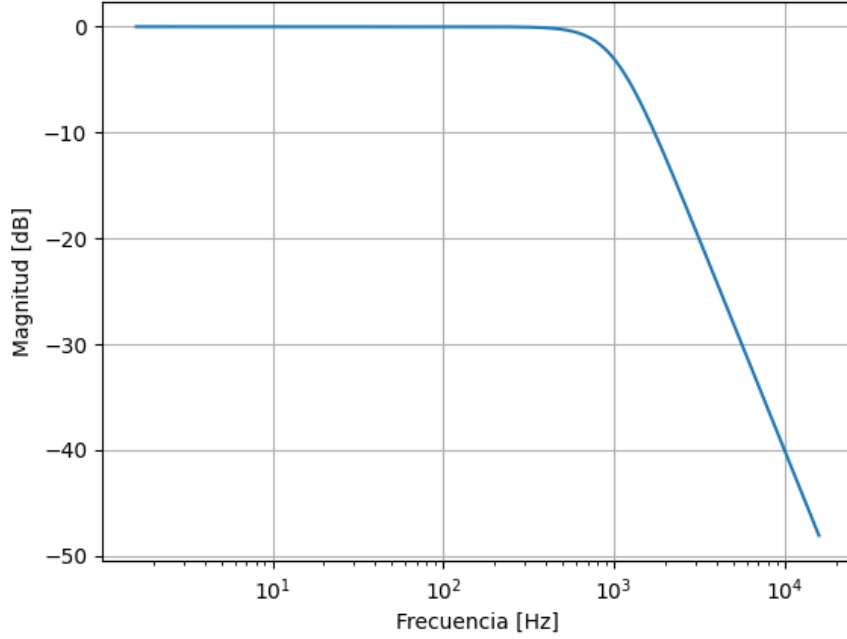


Figura 2: Diagrama de Bode del Ejemplo 2.

Aquí obtendremos una noción mas tangible de la ubicación de los ceros y polos y nos permite utilizar herramientas de análisis para funciones de transferencia, tales como la aplicación del criterio de estabilidad de Routh-Hurwitz o la gráfica del lugar de raíces del sistema por ejemplo.

- Secciones de 2^{do} orden: Aquí se separa la función de transferencia mostrada en (1) en distintas bicuadráticas. Para ello, se hace uso internamente de la función `signal.tf2zpk()` y agrupa los polos y ceros complejos conjugados donde cada par de polos complejos conjugados representan una sección de segundo orden y para los ceros realiza la distinción si son complejos conjugados o reales. En síntesis se obtiene una función de transferencia del estilo:

$$H_i(s) = \frac{b_{0,i}s^2 + b_{1,i}s + b_{2,i}}{s^2 + a_{1,i}s + a_{2,i}} \quad (2)$$

La obtención de una función de transferencia de 2^{do} orden resulta fundamental ya que los filtros no pueden superar este orden, tanto en el denominador debido a que se llegan a los límites de estabilidad del amplificador, tal así como el numerador no puede exceder el grado del denominador por cuestiones de realizabilidad de redes. Con esta función se puede sintetizar un filtro tal así como se realizo en el Trabajo Práctico N°4.

Esta función devuelve un array de 6 elementos, los cuales representan los coeficientes

de la función de transferencia descrita en (2). Nótese que *SciPy* siempre normaliza el termino $a_0 = 1$, por lo que debe ser tenido en cuenta cuando se obtienen los coeficientes de las bicuadráticas.

- Diagramas de Bode: Esta librería nos permite también obtener diagramas de bode, los cuales en nuestro contexto resultan útiles para la verificación de que las frecuencias de corte sean las indicadas en el diagrama de módulo, y de los desfases en el tiempo que le ocurren a las señales de entrada al entrar al filtro en el diagrama de fase.

Además, aquí podemos analizar la estabilidad del amplificador operacional, ya que este al no ser ideal, tiene en su interior una respuesta en frecuencia propia que puede alterar el comportamiento del filtro.

Para ello, se utiliza la función *signal.bode()* la cual toma como parámetros una función de transferencia, y devuelve un array de 3 elementos que representan la frecuencia (en radianes sobre segundo), la magnitud y la fase para poder armar los diagramas de bode.

2.3. Ejemplo de Aplicación en Python

Veamos a modo de conclusión general sobre el tema, un ejemplo de diseño de filtro pasa banda con frecuencias de corte en: $f_1 = 500$ [Hz] y $f_2 = 1500$ [Hz] para el cual se le realizó una aproximación con Butterworth. El código utilizado fue el siguiente:

```
1  import numpy as np
2  from scipy import signal
3  import matplotlib.pyplot as plt
4
5
6  ## Frecuencias de Corte:
7  f1 = 500          # Hz
8  f2 = 1500         # Hz
9  w1 = 2 * np.pi * f1
10 w2 = 2 * np.pi * f2
11 Wn = [w1, w2]
12
13
14 ## Orden del Filtro Prototipo Pasa Bajos:
15 N = 2
16
17
```

```

18  ## Aproximación de Filtro Pasa Banda con ButterWorth:
19  num, den = signal.butter(N, Wn, btype='bandpass', analog = True)
20
21
22  ## Obtención de los Ceros, Polos y Constante:
23  z, p, k = signal.tf2zpk(num, den)
24  print("Ceros: ", z)                # 0 ; 0
25  print("Polos: ", p)                # -3116.32 + j7735.93
26                                     # -3116.32 - j7735.93
27                                     # -1326.56 + j3293.05
28                                     # -1326.56 - j3293.05
29  print("Constante: ", k)            # 39478417.60
30
31
32  ## Descomposición en funciones bicuadráticas:
33  sos = signal.tf2sos(num, den)        # [3.94784176e+07 0 0
34                                     # 1 6.23264064e+03 6.95561180e+07]
35  print("Secciones de segundo orden (SOS):") # [1 0 0
36                                     # 1 2.65312524e+03 1.26039498e+07]
37  print(sos)
38
39
40  ## Diagrama de Bode de cada bicuadrática:
41  plt.figure()
42  for i in range(len(sos)):
43      w, mag, _ = signal.bode((sos[i, :3], sos[i, 3:]))
44      plt.semilogx(w/(2*np.pi), mag, label=f'Etapa {i+1}')
45  plt.xlabel('Frecuencia [Hz]')
46  plt.ylabel('Magnitud [dB]')
47  plt.title('Respuesta individual de cada biquad')
48  plt.grid(True)
49  plt.legend()
50  plt.show()
51
52
53  ## Respuesta en Frecuencia:
54  w, mag, phase = signal.bode((num, den))
55
56  # Gráfico de Magnitud

```

```

57 plt.figure()
58 plt.semilogx(w / (2*np.pi), mag)
59 plt.xlabel('Frecuencia [Hz]')
60 plt.ylabel('Magnitud [dB]')
61 plt.title('Diagrama de Bode - Módulo')
62 plt.grid(True)
63
64 # Gráfico de Fase
65 plt.figure()
66 plt.semilogx(w / (2*np.pi), phase)
67 plt.xlabel('Frecuencia [Hz]')
68 plt.ylabel('Fase [°]')
69 plt.title('Diagrama de Bode - Fase')
70 plt.grid(True)
71 plt.show()

```

De este código, se deducen los siguientes diagramas de bode de módulo y fase para el filtro:

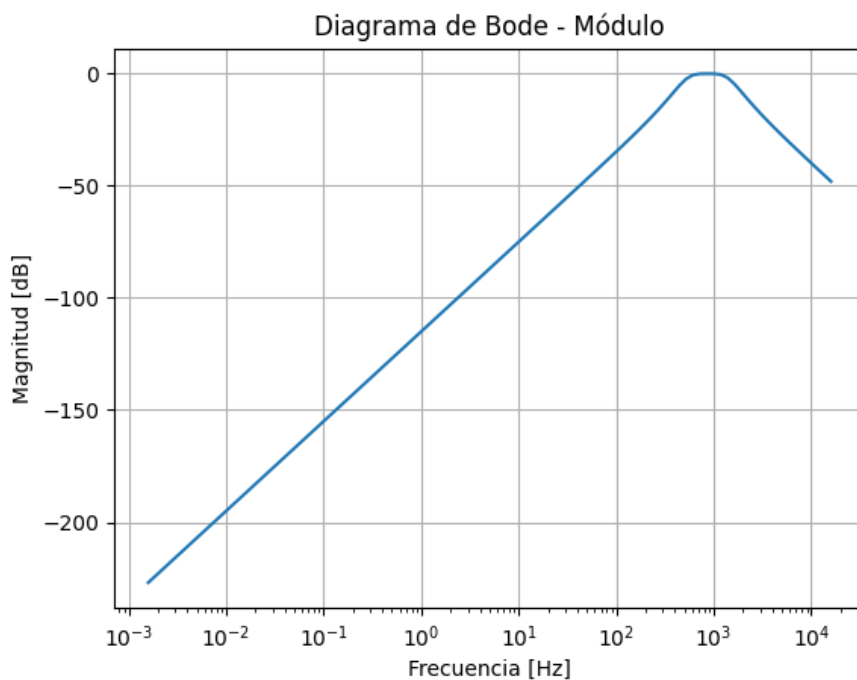


Figura 3: Diagrama de Bode - Módulo del Ejemplo 3 del Filtro Pasa Banda

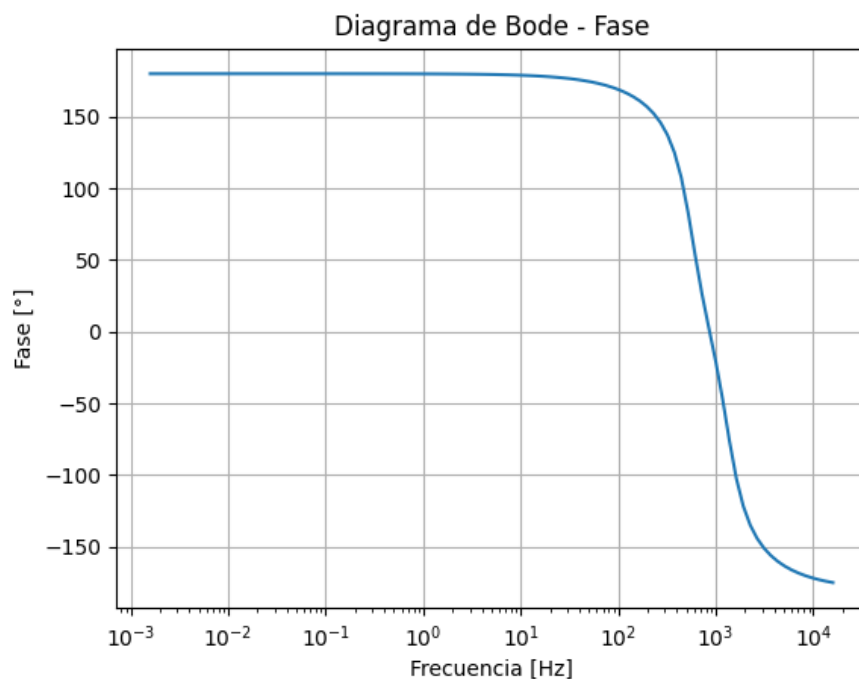


Figura 4: Diagrama de Bode - Fase del Ejemplo 3 del Filtro Pasa Banda

Por último, se presenta la respuesta en frecuencia individual de cada una de las bicuadráticas en su diagrama de bode de módulo:

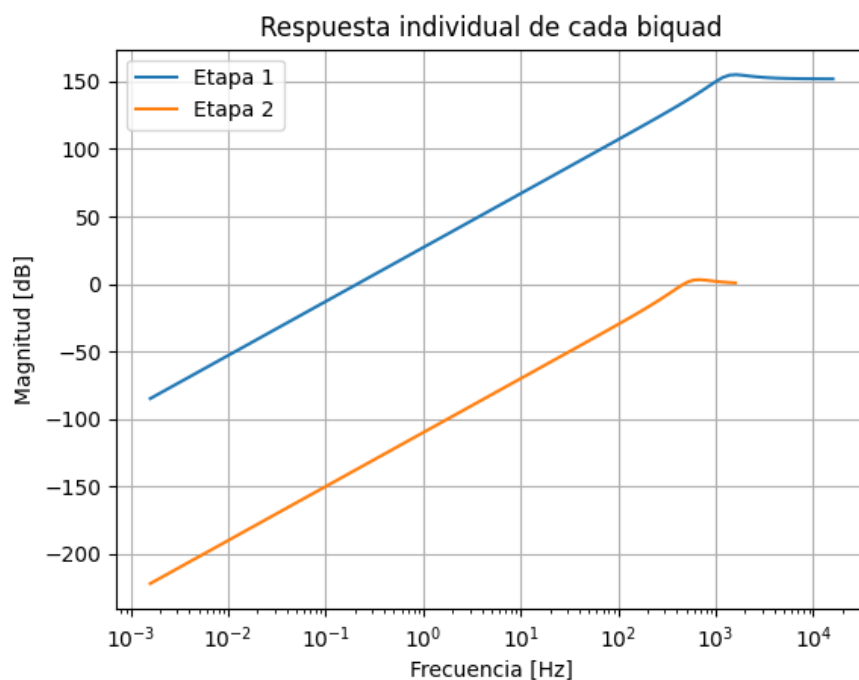


Figura 5: Diagrama de Bode - Módulo del Ejemplo 3 de cada bicuadrática

Se observa aquí la naturaleza propia del algoritmo, el cual genera 2 filtros pasa bajos tal como se espera, ya que si bien se eligió un filtro pasa banda, esta librería parte de un

filtro prototipo pasa bajo el cual luego le realiza una transformación para obtener el filtro deseado. La multiplicación de ambas etapas, nos dará el filtro pasa banda deseado, el cual seguirá el diagrama de bode presentado en la Figura 3.

3. Conclusión

A lo largo de este informe, se realizó una detallada explicación sobre el funcionamiento de las librerías de Python dedicadas a la obtención de funciones de transferencia de filtros dadas ciertas especificaciones del mismo. También se realizaron múltiples ejemplos de diseño para entender como es que funcionan y como realizar este proceso utilizando Python con todos los algoritmos que incluye esta librería.

4. Bibliografía

- Principles of Active Network Synthesis and Design - G. Daryanani
- Repositorio de GitHub con los códigos y demás archivos de este informe
- Documentación del módulo signal de la librería de SciPy

5. Anexo

- Anexo 1: Extracto de la librería SciPy:

```
1 def iirfilter(N, Wn, rp=None, rs=None, btype='band', analog=False,
2             ftype='butter', output='ba', fs=None):
3     """
4     IIR digital and analog filter design given order and critical points.
5
6     Design an Nth-order digital or analog filter and return the filter
7     coefficients.
8
9     Parameters
10    -----
11    N : int
12        The order of the filter.
13    Wn : array_like
14        A scalar or length-2 sequence giving the critical frequencies.
15
16        For digital filters, `Wn` are in the same units as `fs`. By default,
```

17 *`fs` is 2 half-cycles/sample, so these are normalized from 0 to 1,*
 18 *where 1 is the Nyquist frequency. (`Wn` is thus in*
 19 *half-cycles / sample.)*
 20
 21 *For analog filters, `Wn` is an angular frequency (e.g., rad/s).*
 22
 23 *When Wn is a length-2 sequence, ``Wn[0]`` must be less than ``Wn[1]``.*
 24 *rp : float, optional*
 25 *For Chebyshev and elliptic filters, provides the maximum ripple*
 26 *in the passband. (dB)*
 27 *rs : float, optional*
 28 *For Chebyshev and elliptic filters, provides the minimum attenuation*
 29 *in the stop band. (dB)*
 30 *btype : {'bandpass', 'lowpass', 'highpass', 'bandstop'}, optional*
 31 *The type of filter. Default is 'bandpass'.*
 32 *analog : bool, optional*
 33 *When True, return an analog filter, otherwise a digital filter is*
 34 *returned.*
 35 *ftype : str, optional*
 36 *The type of IIR filter to design:*
 37
 38 *- Butterworth : 'butter'*
 39 *- Chebyshev I : 'cheby1'*
 40 *- Chebyshev II : 'cheby2'*
 41 *- Cauer/elliptic: 'ellip'*
 42 *- Bessel/Thomson: 'bessel'*
 43
 44 *output : {'ba', 'zpk', 'sos'}, optional*
 45 *Filter form of the output:*
 46
 47 *- second-order sections (recommended): 'sos'*
 48 *- numerator/denominator (default) : 'ba'*
 49 *- pole-zero : 'zpk'*
 50
 51 *In general the second-order sections ('sos') form is*
 52 *recommended because inferring the coefficients for the*
 53 *numerator/denominator form ('ba') suffers from numerical*
 54 *instabilities. For reasons of backward compatibility the default*
 55 *form is the numerator/denominator form ('ba'), where the 'b'*

56 and the 'a' in 'ba' refer to the commonly used names of the
57 coefficients used.

58

59 Note: Using the second-order sections form ('sos') is sometimes
60 associated with additional computational costs: for
61 data-intense use cases it is therefore recommended to also
62 investigate the numerator/denominator form ('ba').

63

64 *fs* : float, optional
65 The sampling frequency of the digital system.

66

67 .. versionadded:: 1.2.0

68

69 Returns

70 -----

71 *b*, *a* : ndarray, ndarray
72 Numerator (*b*) and denominator (*a*) polynomials of the IIR filter.
73 Only returned if ``output='ba'``.

74 *z*, *p*, *k* : ndarray, ndarray, float
75 Zeros, poles, and system gain of the IIR filter transfer
76 function. Only returned if ``output='zpk'``.

77 *sos* : ndarray
78 Second-order sections representation of the IIR filter.
79 Only returned if ``output='sos'``.

80

81 See Also

82 -----

83 *butter* : Filter design using order and critical points
84 *cheby1*, *cheby2*, *ellip*, *bessel*
85 *buttord* : Find order and critical points from passband and stopband spec
86 *cheb1ord*, *cheb2ord*, *ellipord*
87 *iirdesign* : General filter design using passband and stopband spec

88

89 Notes

90 -----

91 The ``'sos'`` output parameter was added in 0.16.0.

92

93 Examples

94 -----

```

95      Generate a 17th-order Chebyshev II analog bandpass filter from 50 Hz to
96      200 Hz and plot the frequency response:
97
98      >>> import numpy as np
99      >>> from scipy import signal
100     >>> import matplotlib.pyplot as plt
101
102     >>> b, a = signal.iirfilter(17, [2*np.pi*50, 2*np.pi*200], rs=60,
103     ...                          btype='band', analog=True, ftype='cheby2')
104     >>> w, h = signal.freqs(b, a, 1000)
105     >>> fig = plt.figure()
106     >>> ax = fig.add_subplot(1, 1, 1)
107     >>> ax.semilogx(w / (2*np.pi), 20 * np.log10(np.maximum(abs(h), 1e-5)))
108     >>> ax.set_title('Chebyshev Type II bandpass frequency response')
109     >>> ax.set_xlabel('Frequency [Hz]')
110     >>> ax.set_ylabel('Amplitude [dB]')
111     >>> ax.axis((10, 1000, -100, 10))
112     >>> ax.grid(which='both', axis='both')
113     >>> plt.show()
114
115     Create a digital filter with the same properties, in a system with
116     sampling rate of 2000 Hz, and plot the frequency response. (Second-order
117     sections implementation is required to ensure stability of a filter of
118     this order):
119
120     >>> sos = signal.iirfilter(17, [50, 200], rs=60, btype='band',
121     ...                          analog=False, ftype='cheby2', fs=2000,
122     ...                          output='sos')
123     >>> w, h = signal.freqz_sos(sos, 2000, fs=2000)
124     >>> fig = plt.figure()
125     >>> ax = fig.add_subplot(1, 1, 1)
126     >>> ax.semilogx(w, 20 * np.log10(np.maximum(abs(h), 1e-5)))
127     >>> ax.set_title('Chebyshev Type II bandpass frequency response')
128     >>> ax.set_xlabel('Frequency [Hz]')
129     >>> ax.set_ylabel('Amplitude [dB]')
130     >>> ax.axis((10, 1000, -100, 10))
131     >>> ax.grid(which='both', axis='both')
132     >>> plt.show()
133

```



```

134     """
135     fs = _validate_fs(fs, allow_none=True)
136     ftype, btype, output = (x.lower() for x in (ftype, btype, output))
137     Wn = asarray(Wn)
138     if fs is not None:
139         if analog:
140             raise ValueError("fs cannot be specified for an analog filter")
141             Wn = Wn / (fs/2)
142
143     if np.any(Wn <= 0):
144         raise ValueError("filter critical frequencies must be greater than 0")
145
146     if Wn.size > 1 and not Wn[0] < Wn[1]:
147         raise ValueError("Wn[0] must be less than Wn[1]")
148
149     try:
150         btype = band_dict[btype]
151     except KeyError as e:
152         raise ValueError(f"'{btype}' is an invalid bandtype for filter.") from e
153
154     try:
155         typefunc = filter_dict[ftype][0]
156     except KeyError as e:
157         raise ValueError(f"'{ftype}' is not a valid basic IIR filter.") from e
158
159     if output not in ['ba', 'zpk', 'sos']:
160         raise ValueError(f"'{output}' is not a valid output form.")
161
162     if rp is not None and rp < 0:
163         raise ValueError("passband ripple (rp) must be positive")
164
165     if rs is not None and rs < 0:
166         raise ValueError("stopband attenuation (rs) must be positive")
167
168     # Get analog lowpass prototype
169     if typefunc == buttap:
170         z, p, k = typefunc(N)
171     elif typefunc == besslap:
172         z, p, k = typefunc(N, norm=bessel_norms[ftype])

```

```

173     elif typefunc == cheblap:
174         if rp is None:
175             raise ValueError("passband ripple (rp) must be provided to "
176                               "design a Chebyshev I filter.")
177         z, p, k = typefunc(N, rp)
178     elif typefunc == cheb2ap:
179         if rs is None:
180             raise ValueError("stopband attenuation (rs) must be provided to "
181                               "design an Chebyshev II filter.")
182         z, p, k = typefunc(N, rs)
183     elif typefunc == ellipap:
184         if rs is None or rp is None:
185             raise ValueError("Both rp and rs must be provided to design an "
186                               "elliptic filter.")
187         z, p, k = typefunc(N, rp, rs)
188     else:
189         raise NotImplementedError(f"'{ftype}' not implemented in iirfilter.")
190
191     # Pre-warp frequencies for digital filter design
192     if not analog:
193         if np.any(Wn <= 0) or np.any(Wn >= 1):
194             if fs is not None:
195                 raise ValueError("Digital filter critical frequencies must "
196                                   f"be 0 < Wn < fs/2 (fs={fs} -> fs/2={fs/2})")
197             raise ValueError("Digital filter critical frequencies "
198                               "must be 0 < Wn < 1")
199         fs = 2.0
200         warped = 2 * fs * tan(pi * Wn / fs)
201     else:
202         warped = Wn
203
204     # transform to lowpass, bandpass, highpass, or bandstop
205     if btype in ('lowpass', 'highpass'):
206         if np.size(Wn) != 1:
207             raise ValueError('Must specify a single critical frequency Wn '
208                               'for lowpass or highpass filter')
209
210         if btype == 'lowpass':
211             z, p, k = lp2lp_zpk(z, p, k, wo=warped)

```

```

212         elif btype == 'highpass':
213             z, p, k = lp2hp_zpk(z, p, k, wo=warped)
214     elif btype in ('bandpass', 'bandstop'):
215         try:
216             bw = warped[1] - warped[0]
217             wo = sqrt(warped[0] * warped[1])
218         except IndexError as e:
219             raise ValueError('Wn must specify start and stop frequencies for '
220                               'bandpass or bandstop filter') from e
221
222         if btype == 'bandpass':
223             z, p, k = lp2bp_zpk(z, p, k, wo=wo, bw=bw)
224         elif btype == 'bandstop':
225             z, p, k = lp2bs_zpk(z, p, k, wo=wo, bw=bw)
226     else:
227         raise NotImplementedError(f"'{btype}' not implemented in iirfilter.")
228
229     # Find discrete equivalent if necessary
230     if not analog:
231         z, p, k = bilinear_zpk(z, p, k, fs=fs)
232
233     # Transform to proper out type (pole-zero, state-space, numer-denom)
234     if output == 'zpk':
235         return z, p, k
236     elif output == 'ba':
237         return zpk2tf(z, p, k)
238     elif output == 'sos':
239         return zpk2sos(z, p, k, analog=analog)

```